

CONTENIDO DEL PROYECTO

Agente de Análisis de Riesgo Financiero

1. ¿Qué problema resuelve?

Un analista financiero recibe estados financieros extensos en PDF. Debe localizar cifras, calcular ratios, detectar alertas, interpretar información técnica y responder preguntas sobre el documento. Este proceso consume tiempo y puede generar errores. Nuestro agente automatiza una primera evaluación financiera de forma explicable y trazable.

2. ¿Qué hace el sistema?

El sistema recibe un PDF financiero y ejecuta este flujo:

PDF → Extracción de texto → Datos financieros → Indicadores → Alertas → RAG y contexto → Respuesta del agente → SQLite y MLflow

El resultado muestra el nivel de riesgo, los indicadores, las alertas, las recomendaciones y las fuentes utilizadas.

3. Estructura del proyecto

- **backend/**: Lógica y API del sistema
- **frontend/**: Interfaz Streamlit
- **tests/**: Pruebas automatizadas
- **data/**: PDFs sintéticos y base SQLite
- **entregables/**: PPT y material de exposición

4. Lectura del PDF

Archivo principal: [backend/agent/pdf_reader.py](#)

Utiliza PyMuPDF para abrir el PDF y extraer el texto de todas sus páginas. Busca cifras etiquetadas como:

- Activos, Pasivos, Patrimonio
- Ingresos, Utilidad neta
- Efectivo, Inventarios

- Contingencias, Valores del periodo anterior

Si el PDF no existe o es inválido, se genera un error controlado. Si faltan cifras, los indicadores correspondientes quedan como no evaluables.

5. Indicadores financieros

Archivo: [backend/agent/indicadores.py](#)

Calcula los indicadores usando los valores extraídos del PDF: Liquidez corriente, Prueba ácida, Endeudamiento, Ratio de solvencia, Margen neto, ROA, ROE, Variación de ingresos, Contingencias.

```
{
  "nombre": "Liquidez corriente",
  "formula": "Activos corrientes / Pasivos corrientes",
  "valor": 1.85,
  "interpretacion": "Capacidad de cubrir obligaciones de corto plazo.",
  "nivel_riesgo": "bajo"
}
```

El cálculo maneja división entre cero y datos faltantes sin detener el sistema.

6. Alertas

Archivo: [backend/agent/alertas.py](#)

Las alertas se generan automáticamente mediante reglas configurables. Cada alerta contiene: Indicador, Valor, Umbral, Severidad, Descripción, Recomendación.

Las severidades son: BAJA, MEDIA, ALTA, CRÍTICA. Los umbrales están centralizados en DEFAULT_ALERT_RULES, por lo que pueden modificarse sin cambiar el resto del sistema.

7. RAG (Recuperación Aumentada de Información)

Archivo: [backend/agent/rag.py](#)

El sistema:

- Divide el texto en fragmentos o chunks.
- Convierte los fragmentos en embeddings TF-IDF.
- Convierte la pregunta en un vector.
- Calcula similitud coseno.
- Devuelve los fragmentos más relevantes.

Estos fragmentos se utilizan como contexto para responder la pregunta y se muestran como fuente. La implementación actual es local y no requiere una base vectorial externa.

8. Cliente Groq

Archivo: [backend/agent/llm_client.py](#)

Todas las llamadas al modelo pasan por este archivo. Utiliza: `from groq import Groq`. La clave se obtiene mediante: `os.environ.get("GROQ_API_KEY")`

Los modos de respuesta son:

- **Rápido:** GPT-OSS 20B.
- **Equilibrado:** GPT-OSS 120B con razonamiento medio.
- **Avanzado:** GPT-OSS 120B con razonamiento alto.

Las respuestas del chat se transmiten mediante streaming.

9. LangGraph

Archivos: [backend/workflow/state.py](#), [backend/workflow/graph.py](#), [backend/workflow/nodes/](#)

El proyecto utiliza un StateGraph de LangGraph. El flujo es:

START → extractor_node → calculadora_node → alertas_node → respuesta_node → END

El FinancialState transporta la información entre nodos (Ruta, Texto, Datos, Indicadores, Alertas, Contexto, Riesgo, Respuesta, Errores). Los nodos son delgados y delegan la lógica en `backend/agent/`.

10. FastAPI

Archivo: [backend/main.py](#)

Endpoints:

- GET `/health`
- POST `/analizar` (Recibe PDF, ejecuta LangGraph y devuelve análisis. Lo guarda en SQLite).
- POST `/preguntar` (Recibe file, pregunta, modo. Devuelve respuesta por streaming y termina con la fuente).

11. SQLite & 12. MLflow

- **SQLite** ([backend/agent/storage.py](#)): Guarda cada análisis en `data/analyses.db` (Fecha, Nombre, Riesgo, Indicadores, Alertas, Respuesta).
- **MLflow** ([backend/observability/tracing.py](#)): Registra localmente en `mlruns/` la ejecución del grafo, tiempos de los nodos, entradas y salidas resumidas. No registra el texto completo innecesariamente.

13. Frontend

Archivo: [frontend/app.py](#)

Desarrollado con Streamlit. Tiene dos secciones:

- **Análisis:** Subir PDF y visualizar riesgo, indicadores, alertas, recomendaciones y RAG.
- **Preguntas:** Elegir modo, preguntar, ver respuesta progresiva y modo utilizado (Logo TCS y diseño corporativo).

14. Pruebas

La suite cubre: Indicadores, Alertas, RAG, Lectura de PDFs, API, Preguntas, SQLite, MLflow.

Última validación: 24 tests ejecutados | 24 tests aprobados.

15. Guion corto para la exposición (Elevator Pitch)

"Nuestro proyecto es un Agente de Análisis de Riesgo Financiero. El sistema recibe un estado financiero en PDF, extrae sus cifras principales, calcula indicadores, detecta alertas y permite hacer preguntas sobre el documento.

Utilizamos LangGraph para orquestar el flujo mediante un StateGraph. Cada nodo tiene una responsabilidad independiente: extracción, cálculo, alertas y respuesta.

Para las preguntas usamos RAG local con TF-IDF y similitud coseno. Esto permite recuperar fragmentos relevantes del PDF y citar la fuente utilizada.

Las respuestas se generan mediante Groq y llegan por streaming. Los análisis se guardan en SQLite y las ejecuciones se registran en MLflow."

16. Demostración

1. Abrir `http://localhost:8501` y subir `data/ejemplo_financiero.pdf`.
2. Ir a Análisis → Pulsar "Analizar documento".
3. Mostrar nivel de riesgo, indicadores, alertas y recomendaciones.
4. Ir a Preguntas → Elegir Equilibrado.
5. Preguntar: *¿Cuál es el nivel de endeudamiento de la empresa?*
6. Mostrar la respuesta progresiva y la fuente recuperada.

17. Preguntas frecuentes

- **¿Los datos son reales?** No. Se utilizan PDFs sintéticos.
- **¿Qué pasa si falta información?** El sistema evita inventar cifras. El indicador se marca como no evaluable.
- **¿Por qué usar LangGraph?** Permite representar el proceso como un grafo de estados, con nodos independientes.
- **¿Por qué SQLite / MLflow?** SQLite para persistir análisis; MLflow para observar y medir tiempos.
- **¿Listo para producción?** Es una primera versión. Faltaría calibrar umbrales, mejorar extracción, reforzar seguridad y validar con expertos financieros.

18. Comandos de ejecución

Backend:

```
.venv\Scripts\Activate.ps1
uvicorn backend.main:app --reload --port 8001
```

Frontend:

```
.venv\Scripts\Activate.ps1
streamlit run frontend/app.py
```

Tests:

```
pytest tests/ -v
```

Compilación:

```
python -m compileall -q backend frontend
```